Uncovering Bigger Truths: Deobfuscating PHP with Phoebe

Manuel Karl
TU Braunschweig
m.karl@tu-braunschweig.de

Simon Koch†
University of Innsbruck
simon.koch@uibk.ac.at

David Klein
TU Braunschweig
david.klein@tu-braunschweig.de

Martin Johns
TU Braunschweig
m.johns@tu-braunschweig.de

***Abstract*—Code obfuscation is especially prominent in server-side scripting languages. For instance, almost all webshells — backdoors installed by attackers to gain persistent access to a hacked system — and similar PHP-based malware are heavily obfuscated to hide their logic and true nature. Deobfuscation is the ability to reverse code obfuscation, i.e., to revert an obfuscated program into a form as close as possible to the original, unknown input, without changing its semantics. This is essential for incident response teams and developers alike to understand foreign code, assess how malicious programs work, and gather clues about the perpetrators.**

In this work, we focus on the challenges specific to PHP deobfuscation. To do so, we first study ten PHP obfuscators to assess how they obfuscate code by identifying and isolating the transformations they employ. Based on these insights, we propose Phoebe, a deterministic deobfuscator that statically reverses PHP obfuscation. We built a large dataset of PHP files sampled from popular open-source applications and their obfuscated versions to showcase Phoebe's efficacy. We then deobfuscate this dataset with both Phoebe and two other best-in-class PHP deobfuscators. We assess the results based on syntactic correctness, similarity, and code complexity. While Phoebe is the only deobfuscator that does not cause syntax errors, it also retrieves files that resemble the original file by 80% similarity, outperforming the competition by over 40%.

***Index Terms*—PHP, Deobfuscation, Forensics**

1. Introduction

PHP has a dominating market share among server-side programming languages [42]. It is an interpreted scripting language, which makes both deployments and installation of add-ons trivial. Copying the PHP scripts into a directory readable by the web server is sufficient to start serving your application. Similarly, updating or extending an application only requires overwriting or adding new files to the folder. This is frequently done directly by the application; i.e., it allows uploading PHP files that are executed later. One prominent example of this file handling pattern is WordPress, the best-known PHP application and the most widely used content management system on the web, used by 43.4% of all websites [43]. The security implications of such file handling practices are emphasized by Kasturi et al. [19], who analyzed 3452 WordPress plugins and found that 16% of the plugins contained obfuscated code. Furthermore, it opens the door for web shells, as shown by the analysis, which revealed that 28% of the analyzed plugins included web shells. Here, an attacker compromises a server and places a PHP script that opens up functionality, such as shell access, to the outside world, allowing for persistent access. This can be achieved by exploiting a file upload vulnerability, such as updating the application or installing plugins or themes. Attackers almost always use code obfuscation in their web shell code to hide their tracks [38].

Code obfuscation is a method of modifying code to retain its semantics while being almost impossible for a human to analyze. This includes mangling identifiers, modifying and bloating the control flow with numerous `goto` statements, and placing the code inside a string that has several layers of encoding applied before calling the corresponding decoding functions at runtime, and then dynamically executing the result via `eval`. Another example of obfuscation usage is phishing kits. Here, applications, often written in PHP, are sold to people aiming to launch phishing campaigns. Cova et al. [9] studied such phishing kits and found they often employ obfuscation to hide backdoors, exfiltrating data back to their vendor.

Therefore, analyzing code is crucial, for example, in identifying vulnerabilities in web applications or understanding the extent of a compromise in the case of a web shell. Analyzing a web shell can also reveal insights about the attacker, which can help mitigate further attacks and compromises. To do so, the goal is to reverse the code obfuscation, also known as deobfuscation. The perfect deobfuscator would return the original code for any program p, i.e., $deobfuscate(obfuscate(p)) = p$. While this ideal is not achievable, as some obfuscations are destructive, i.e., by changing identifiers and layout, it is desirable to return code that resembles the original as closely as possible. In this work, we set out to achieve this for PHP deobfuscation.

To achieve this, we first analyzed how ten PHP obfuscators transform code. Thereby, we identified nine different obfuscation patterns, ranging from mangling identifiers to obfuscating the control flow by replacing regular program flow with a vast number of `goto` statements, i.e., jumps. Based on these patterns, we propose Phoebe, a fully de-

†. Parts of this research were conducted at TU Braunschweig.

terministic and static PHP deobfuscator that can reverse all nondestructive obfuscation patterns we encountered. Phoebe is capable of reversing multiple layers of different encodings or even complex control flow obfuscation, such as recovering loop structures based on jumps.

To assess the efficacy of Phoebe, we evaluate it against six obfuscators and compare it against two alternative PHP deobfuscators. Phoebe is the only deobfuscator without syntax errors and able to achieve over 80% code similarity to the original files before obfuscation. This is significantly better than the other deobfuscators that achieve 0.26% and 0.40% code similarity while creating dozens of syntax errors. Consequently, Phoebe opens the door to analyzing malicious web shells and phishing kits reliably for future work.

In summary, we make the following contributions:

- We analyze ten PHP obfuscators and distill nine distinct obfuscation patterns.
- We build Phoebe, a deterministic deobfuscator that reverses PHP obfuscation fully statically.
- We build a large dataset consisting of original and obfuscated code versions and evaluate Phoebe in a large-scale evaluation against two best-in-class deobfuscators, outperforming them both significantly.

2. PHP Obfuscation Patterns

Effective deobfuscation starts with understanding common obfuscation patterns. We identified in-scope obfuscators, excluding those using encryption with external key materials or modified PHP interpreters. This yielded ten obfuscators, which we analyzed in depth; Table 1 summarizes their availability and techniques. We found that they employ a total of nine distinct obfuscation patterns, which we detail in the following.

2.1. Identifier Mangling (IM)

One of the most common obfuscation patterns against human analysis is renaming class, function, and variable names, as shown in Listing 1. Explicit and descriptive names are essential for developers who want to analyze, extend, or debug code. Losing this information provided by the original developer hinders humans' understanding of the code, but it does not affect automated code analysis.

We encountered eight obfuscators that employ identifier mangling. For example, Obfuscator-Class and Smart-PHP-Obfuscator add code used for decoding with vacuous identifiers, such as `$__` or `$_5MNtw`.

```
class Consts {
 const TIMEOUT = 10;
}
function sec($prm) {
 return $prm / 10;
}
sec(Consts::TIMEOUT);
```

→

```
class AfgTDv {
 const osfERTa = 10;
}
function ab4($a3s) {
 return $a3s / 10;
}
ab4(AfgTDv::osfERTa);
```

Listing 1: Identifier Mangling of variables and functions.

While Pipsomania, Naneau, and PHP Obfuscator only rename strings and local-scope variables, such as loop indices, Gaijn, YAK Pro, and PHP-Einfach replace any variable or function name with a random string.

2.2. Formatting (F)

Similar to identifiers, the code's layout is an important aspect aiding humans in its understanding. We encountered seven that deliberately remove formatting or add excessive whitespaces to destroy any resemblance of formatting. In particular, inserting a large number of line breaks while combining several statements in the same line without spaces makes it difficult to understand the code at a glance.

2.3. Data Hiding (DH)

Three obfuscators employ Data Hiding, an obfuscation strategy that aims to disguise constant values. Instead of keeping the actual values, Data Hiding splits them into several small pieces and reconstructs them at runtime, using string manipulation operations, as shown in Listing 2. Common patterns use `substr`, `strrev`, and string concatenation to recreate strings by joining separate parts. This makes it difficult for humans to understand the code, as they have to follow the sequence of string operations to restore the constants guiding the program.

```
$a = "SEC";
```

→

```
$L = "XSCETGVIL";
$a = strrev(substr($L, 2, 2) .
  substr($L, 1, 1));
```

Listing 2: Data Hiding to obfuscate string constant.

2.4. Encoding (ENC)

While a human can usually still decode data hiding, encoding goes a step further and prevents this by changing the representation of string data into a non-human-readable format. We encountered eight obfuscators employing some form of encoding. As the PHP standard library provides a wide range of encoding functions, all obfuscators employ decoding using these, such as `rot13`, or `base64`. PHP-Einfach, and Mobilefish, also employ compression such as `gzip`. Obfuscator-Class and Smart-PHP-Obfuscator even take it one step further and use custom encoding algorithms. In this case, the obfuscator injects additional code into the file to provide the decoding routines; i.e., shipping its decoding runtime. Listing 3 showcases encoding-based obfuscation, employing both built-in and custom encodings.

```
print_r("test");
```

→

```
function dec($str) {
 return str_rot13(
   substr($str,1,7));
}
dec("acevag_ea")(
 base64_decode(...));
```

Listing 3: Application of custom and built-in encodings.

TABLE 1: All investigated obfuscators with their access type and identified patterns (IM: Identifier Mangling, Data: Data Hiding, ENC: Encoding, DS: Data Scattering, DCE: Dynamic Code Execution, F: Formatting, LSO: Logical Structure Obfuscation, IN: If-Negation, BBM: Basic Block Mixing, LD: Loop Deconstruction)

Obfuscator	ID	Accesss Type	IM	DH	ENC	DS	DCE	F	LSO		
									BBM	IN	LD
Obfuscator-Class [37]	OBF 1	offline	✓	✓	✓	✓	✓	✓	✗	✗	✗
Smart-PHP-Obfuscator [45]	OBF 2	offline	✓	✓	✓	✓	✓	✓	✗	✗	✗
YAK Pro [21]	OBF 3	offline	✓	✗	✓	✗	✓	✗	✓	✓	✓
Naneau [12]	OBF 4	offline	✓	✗	✗	✗	✗	✓	✗	✗	✗
PHP-Obf† [1]	OBF 5	online	-	-	-	-	-	-	-	-	-
PHP Obfuscator [10]	OBF 6	online	✓	✗	✓	✗	✓	✓	✓	✗	✗
PHP-Einfach [31]	OBF 7	online	✓	✗	✓	✗	✓	✓	✗	✗	✗
Gaijn [32]	OBF 8	online	✓	✗	✓	✗	✗	✓	✗	✗	✗
Pipsomania [6]	OBF 9	online*	✓	✗	✓	✓	✗	✓	✗	✗	✗
Mobilefish [27]	OBF 10	online*	✗	✓	✓	✗	✓	✗	✗	✗	✗

* Protected by Captcha, † Broken at the time of evaluation.

2.5. Data Scattering (DS)

Data Scattering is an obfuscation technique that pulls the definition of constants or functions away from their usage. Since humans have a limited short-term memory capacity, frequent navigation in the source file complicates code analysis. We encountered three obfuscators that employ this technique, as shown in Listing 4.

```
// .. stuff ..
$a = compSqrt(42);
```
→
```
$fnc = "compSqrt";
$param = 42;
// .. stuff ..
$a = $fnc($param);
```

Listing 4: Scattering of constants.

2.6. Dynamic Code Execution (DCE)

This pattern describes moving the source code into a string and dynamically evaluating it at runtime with `eval` This is trivial to analyze on its own, as the code remains unchanged. However, storing the program in a string allows combinations with other obfuscation techniques, e.g., encoding, rendering the original code unintelligible. We observed obfuscation by dynamically executing it via `eval` in six obfuscators. Listing 5 shows how wrapping code in `eval` with base64 encoding looks like.

```
function prt($ele) {
  echo $ele;
}
prt(42);
```
→
```
eval(base64_decode(
"ZnVuY3Rpb24\
gcHJ0KCRlbGVtKSB\
7CiAgIGVjaG8gJGVs\
ZW07Cn0gICAgCnByd\
Cg0Mik7"));
```

Listing 5: Dynamically executing code after decoding it.

2.7. Logical Structure Obfuscation (LSO)

Previous obfuscation patterns primarily aimed to conceal code properties, hindering both manual and automated analysis. Despite this, they preserved the program's original execution semantics. Two obfuscators go beyond by employing transformations that substantially alter the code's appearance and control flow while maintaining the program's original semantics. We observed three distinct patterns in this group: YAK Pro employed all, and PHP Obfuscator employed one, namely, basic block mixing.

2.7.1. Basic Block Mixing (BBM). A program can be split according to its execution flow, dividing it into linear control-flow sections, so-called *basic blocks* [2]. This logical unit allows a programmer to keep track of the execution flow and the effect of subsequent instructions.

We encountered two obfuscators that split a single basic block into multiple blocks, each starting with a `label` and ending with a `goto` statement pointing to the next block, as shown in Listing 6. They then mix those smaller logical units, effectively scrambling the statement order, resulting in incomprehensible code.

```
$a = 42;
$a = $a*$a;
echo $a;
return;
```
→
```
goto a;
c: echo $a; goto d;
a: $a = 42; goto b;
d: return;
b: $a = $a*$a; goto c;
```

Listing 6: Basic Block Mixing employing goto jumps

2.7.2. If-Negation (IN). In addition to Basic Block Mixing, we observed a second logical obfuscation pattern: If-Negation, which inverts an if-condition and swaps the if and else branches (Listing 7).

```
if (isset($a)) {
 if ($a < 8) {
  echo "yes";
 } else {
  echo "no";
 }
}
```
→
```
goto ljocr;
aA8D9:
if (!($a < 8)) goto K5;
goto g_EJQ;
ljocr:
if (!isset($a)) goto F;
goto aA8D9;
K5: echo "no"; goto F;
g_EJQ: echo "yes";
F:
```

Listing 7: If negation as employed by YAK Pro

Effectively, this inverts the original control flow, i.e., the logical structure, intended by the developer. Furthermore, all statements are moved outside of either case's block by a jump, disguising which statements belong to which condition.

2.7.3. Loop Deconstruction (LD). The final obfuscation pattern, also unique to YAK Pro, is the deconstruction of loops. YAK Pro takes common loop patterns and deconstructs them into a combination of conditions and jumps, as shown in Listing 8. In combination with Basic Block Mixing, this results in a loop that is distributed across its whole scope, making it unrecognizable.

```
for($i=0;$i<9;$i++)
 echo "iter\n";
```

→

```
$i=0; s:
if(!($i < 9)) goto e;
goto body;
incr: $i++; goto s;
body: echo "iter\n";
goto incr;
e:
```

Listing 8: Loop Deconstruction employed by YAK Pro

3. Static Deobfuscation

The transformations we identified in the last section prevent both human and automated analysis. However, developers and incident response teams need to be able to understand any code they encounter or want to employ reliably. We propose a deobfuscator, called *Phoebe*, to resolve this issue based on this in-depth understanding of real-world obfuscation patterns.

3.1. Code Preparation

Before any deobfuscation, Phoebe starts with a preparation step that is divided into three stages: 1) Parsing: We leverage nikic-parser [30] to parse the obfuscated source code, resulting in a JSON-encoded Abstract Syntax Tree (AST). 2) AST Creation: Then, we turn the JSON AST into an actual graph structure. 3) Control Flow Graph (CFG) Creation: Next, we run a standard CFG generation algorithm on the AST and add control flow edges. Consequently, we go from obfuscated source code towards a graph representation, as shown in Figure 1, on which our deobfuscator operates.

3.2. Deobfuscation

We divided the deobfuscation into separate passes, each addressing one of the obfuscation patterns discussed in Section 2. Every pass is independent of the other passes, resulting in a modular design that allows for straightforward extension, i.e., future-proofing it against novel obfuscation routines. All implemented passes are used in a fixpoint iteration, which runs all passes in sequence until no further changes are made to the current code, indicating that deobfuscation is complete. Out of the nine obfuscation patterns, we address seven with Phoebe. We cannot address Identifier

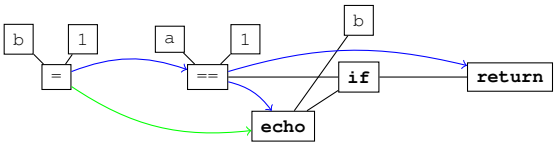


Figure 1: Graph for: `b=1; if(a==1) echo(b); else return;` with edges in black (AST), blue (CFG), and green (DDG).

Mangling (cf. Section 2.1) and Formatting (cf. Section 2.2) as the original names and format are irretrievably lost. However, we restore code in a well-formatted way, following basic indentation rules. We now go over each pass in detail.

Data Hiding. To counter data hiding, we implemented transformation rules to resolve built-in string manipulating functions and their multi-layered combinations. This means that we scan the AST for any function call to a supported routine, i.e., `substr`, `strrev`, `preg_replace`, or string concatenation. If we encounter such a call with static parameters, we apply the transformation rule and replace the call node with its resulting value.

Encoding. Similar to Data Hiding, we handle encoding by applying transformation rules to built-in encoding functions. Therefore, we scan the AST for supported routines, i.e., `base64_decode`, `gzinflate`, `gzuncompress`, `str_rot13`, `hex2bin`, with static values and replace the call node with the transformation's result. Furthermore, we also support custom routines, i.e., functions defined by the obfuscator and injected into the obfuscated code as shown in Listing 3. To deobfuscate such calls, we first ensure that the decoding routine does not reference any external variables, such as globals or superglobals. Then, we perform a separate deobfuscation on a copy of the function, replacing its parameters with the call's static arguments. If the separate fixpoint iteration on the copied function yields a single static return value, we replace the original call to the function with that result.

Data Scattering. To counter Data Scattering, we leverage a Data Dependency Graph (DDG) based on the existing CFG by propagating all constant values based on the DDG edges. As PHP does not differentiate between `func()` and `"func"()`, this also allows us to inline scattered function names. In addition to inlining constant values within functions, we also analyze whether global values are changed after initialization. If the global values remain unchanged, we propagate them, i.e., replacing all references with their value. This enables us to resolve custom encoding functions depending on global values.

Dynamic Code Execution. Phoebe also aims to reverse Dynamic Code Execution, i.e., wrapping the program code in a string and executing it with **`eval`**. Frequently, this occurs with several nested **`eval`** calls, i.e., **`eval`**`(`**`eval`**`(`**`eval`**`(..)))`. To unpack nested **`eval`** calls, the process is as follows. For **`eval`**`(`**`eval`**`(code))`, the outer **`eval`** is a no-op if *code* does not return a string value [13]. Consequently, if the parse tree of *code* does not have a top-level return statement, we can directly delete the outer **`eval`** call. This is done iteratively

until we are left with a single `eval` call. We build a new AST by replacing the `eval` call with the code passed to it. Thus, we effectively inlined the eval, allowing us to deobfuscate the code further.

Basic Block Mixing. To deobfuscate Basic Block Mixing, we leverage our CFG. We collect the set of all existing labels and filter the number of incoming CFG edges. If there is only a single incoming edge, it's a potential candidate for inlining. For these, we check if the incoming edge has a `goto` statement as a predecessor node. If this is the case, we inline the label and subsequent code block just after the corresponding `goto`. Afterward, we remove both the label and the `goto`. If the predecessor is not a `goto`, we remove the label because it is not targeted by any `goto`.

If Negation. To deobfuscate *If Negation*, we scan for `if` statements containing a single `goto` statement, followed by another `goto` statement. Afterward, we invert the condition and swap the `goto` statements. Therefore, the `if` clause jumps to the code originally part of the `if` statement. In this way, we restore the original version of the control flow that the developer intended.

Loop Deconstruction. In Listing 8, we have seen how an obfuscator splits a for loop into multiple basic blocks. We exploit the fact that this split has a clear pattern: 1) the first block contained in the loop is a condition that exits the loop, 2) the last block of the loop does a mathematical operation on a variable, e.g., a counter, 3) the block that precedes the first block of the loop initializes the value of the loop variable. Thus, we identify all unique cycles with a single entry point within our CFG and check if the criteria 1 through 3 are fulfilled. If this is the case, we can reconstruct the original `for` loop by moving each condition into the head of the loop at its corresponding position. In the special case, where none or only criterion 1 is fulfilled, we forgo the `for` loop. We can nevertheless reconstruct a `while` loop, with the condition in the head if criterion 1 is fulfilled or `true` as the condition otherwise.

4. Evaluation

We searched for other PHP deobfuscators to compare the correctness and efficacy of Phoebe. As a starting point, we reviewed related work and identified UnPHP [40], an online PHP deobfuscator, as a state-of-the-art deobfuscator [17, 20, 38, 46]. We subsequently conducted a Google search to find further deobfuscators, yielding PHPDeobfuscator [36], another offline PHP deobfuscator.

To compare Phoebe against both third-party deobfuscators, we generated a large dataset of obfuscated files using six of the ten obfuscators listed in Table 1. We removed the following four deobfuscators: Pipsomania, Mobilefish due to CAPTCHAs preventing automation, PHP-Obf returned bad HTTP status code for any input, and, finally, we removed Naneau as it can not handle basic PHP features such as classes and namespaces, i.e., failing for most of our dataset.

Since UnPHP as well as the three online obfuscators require one web request for every (de)obfuscation, we limited the number of requests due to ethical considerations. Thus, we split the evaluation into two parts: A large-scale study with 95541 obfuscated code samples to compare against PHPDeobfuscator (Section 4.3), and a smaller study with 2921 samples for comparison with UnPHP (Section 4.4). Lastly, to assess Phoebe's impact on real-world malicious inputs, we conducted a small study with webshells collected from GitHub (Section 4.5).

4.1. Evaluation Criteria

Deobfuscation aims to reverse obfuscation and restore the code to a human-readable state. The optimal outcome would be for a deobfuscator to restore the original code. However, as some transformations destroy information, retrieving the original file is impossible, leading us to assess the efficacy using the following criteria. We evaluate the correctness of the deobfuscation process and use three different measures, split into one similarity and two complexity metrics, to evaluate the impact of ob- and deobfuscation.

Errors. Only if the deobfuscator: 1) terminates in a timely fashion, 2) without crashing, 3) and produces a valid PHP file, the result is usable. Therefore, we first analyze any errors occurring in this stage. We set a time limit of five minutes and forcefully stop the deobfuscation once the limit is reached. Crashes are all cases where a deobfuscator terminates unsuccessfully, returning an error, e.g., due to an uncaught exception. We check whether the resulting file is valid PHP on successful termination with `php -l`.

Code Similarity. To understand how well 1) obfuscation hides the original code on a syntactical level, and, 2) a deobfuscator can reconstruct the code, we leverage the code similarity metric proposed by Li et al. [25]. This metric compares two files by measuring the number of syntactical differences. While it reflects complex changes, it also ensures that simply moving pieces of code around does not zero the similarity, e.g., changing `a = 1; b = 2;` to `b = 2; a = 1;` only reduces similarity, as sub-elements of the code are still equal. While it ignores names, i.e., renaming functions or variables has no effect, it considers constant values; that is, scalar nodes with different values are not considered equal. A similarity value of 100% indicates that two compared samples have identical code structure.

We use this metric to calculate both the effect of the obfuscators, when applied to our original dataset, and the effectiveness of subsequent deobfuscation (Table 5 and Table 8), by calculating the similarity of the resulting files and the original ones. In cases where deobfuscation is unsuccessful, such as due to timeouts, errors, or syntax errors in the result, we take the similarity between the obfuscated and the original file as a stand-in value. This ensures fairness, as a user employing a deobfuscator tool only has the obfuscated code available in case of failure. While we would be able to output a partially deobfuscated file in case of a timeout, this option is unavailable for online deobfuscators such as UnPHP. Thus, we consider this the only way to assess deobfuscation efficacy fairly.

TABLE 2: PHPDeobfuscator Dataset Characteristics

Obf.	Size	Cognitive	Cyclomatic
OBF 1	30 769	113.50 (σ=402.12)	82.73 (σ=147.33)
OBF 2	32 356	111.46 (σ=393.22)	81.65 (σ=144.76)
OBF 3	26 708	90.15 (σ=144.50)	70.90 (σ=82.48)
OBF 6	2266	114.68 (σ=226.66)	79.34 (σ=106.25)
OBF 7	2065	83.53 (σ=205.56)	57.70 (σ=119.10)
OBF 8	1377	72.39 (σ=145.01)	52.74 (σ=66.64)

TABLE 3: UnPHP Dataset Characteristics

Obf.	Size	Cognitive	Cyclomatic
OBF 1	500	110.93 (σ=226.58)	81.78 (σ=163.03)
OBF 2	500	125.95 (σ=283.05)	88.01 (σ=135.90)
OBF 3	474	102.07 (σ=139.57)	73.82 (σ=78.60)
OBF 6	491	128.25 (σ=300.42)	87.52 (σ=122.89)
OBF 7	500	76.37 (σ=153.01)	53.20 (σ=78.47)
OBF 8	456	76.11 (σ=136.59)	55.39 (σ=60.22)

Cyclomatic & Cognitive Code Complexity. Deobfuscation is intended to help both tools and human analysts understand the code and its properties. Prior work [16] used cyclomatic complexity to assess deobfuscation efficacy by measuring changes in complexity due to deobfuscation. We evaluate cyclomatic complexity to gauge how well it reflects deobfuscation effectiveness. Additionally, we assess cognitive complexity [4], which aims to address cyclomatic complexity's limitations, to determine whether 1) complexity metrics capture deobfuscation efficacy, and, 2) whether cognitive complexity offers advantages in this context. Even if deobfuscation does not fully recover the original code, it is still valuable if it reduces the mental load required for understanding; the assumption is that changes in the complexity metrics capture this effect. Here we again use the results of the obfuscated file if deobfuscation fails.

Cyclomatic Complexity. Cyclomatic complexity is a traditional metric proposed by McCabe [26]. It assesses the complexity of the control flow graph and reflects the number of independent control flow paths in a program. Therefore, the metric is often used to measure testability and maintainability. Furthermore, this intuitively reflects the difficulty for humans or tools to analyze the underlying code, as numerous independent paths result in a high number of control flow combinations requiring examination.

Cognitive Complexity. Campbell [4] introduced Cognitive Complexity, which focuses on the readability and maintainability of code. Code with a high number of branches can still be easy for humans to understand, despite its high cyclomatic complexity. For example, a `switch` statement introduces one control flow branch per `case`, greatly increasing the cyclomatic complexity. While this impacts testability, as each branch requires testing, it does not incur much cognitive overhead for a human. To counter this shortcoming, cognitive complexity measures code structures that are difficult for humans to understand. In contrast to cyclomatic complexity, `goto` statements are taken into account, as frequent jumping between blocks makes it difficult to follow the program logic. Additionally, this metric takes nesting depth into account, which also causes difficulties for humans. We use this metric to complement cyclomatic complexity and assess whether they are similarly applicable.

4.2. Methodology

Assessing the effect of obfuscation and subsequent deobfuscation requires knowledge of the original code. We sampled our dataset from the top 10 000 PHP GitHub repositories based on stars. We use stars as a surrogate metric to ensure commonly used code [22]. On 11th of December 2024 we randomly selected 33 000 files, drawn from all `.php` files with at least 200 lines of code (excluding comments and whitespace) and no syntax errors.

Since several evaluated tools are available only online, we accounted for their usage impact by reducing the dataset for each online tool. Specifically, we sampled 3000 files from our large dataset for obfuscation using online obfuscators. Finally, we sample 500 successfully obfuscated files from each obfuscator for deobfuscation with UnPHP.

We removed any file for which the evaluation was unsuccessful due to reasons outside the influence of the assessed deobfuscation tools, such as an obfuscator returning a syntactically broken file, or the calculation of our chosen metrics taking longer than five minutes , i.e., timed out.

This resulted in two overarching datasets: One we use to evaluate Phoebe against PHPDeobfuscator, shown in Table 2, and the second used to evaluate against UnPHP, shown in Table 3. For further details on the obfuscation errors and timeouts during the evaluation, see Table 10, Table 11, and Appendix B.

4.3. Comparison with PHPDeobfuscator

We ran our evaluation on a machine with an AMD EPYC 7702P CPU and 512GB of RAM, using a five minutes timeout for deobfuscation.

Runtime Distribution. Figure 2 is a visualization of the runtime distribution for Phoebe across the different input sizes, with the lighter coloring indicating a higher datapoint density.

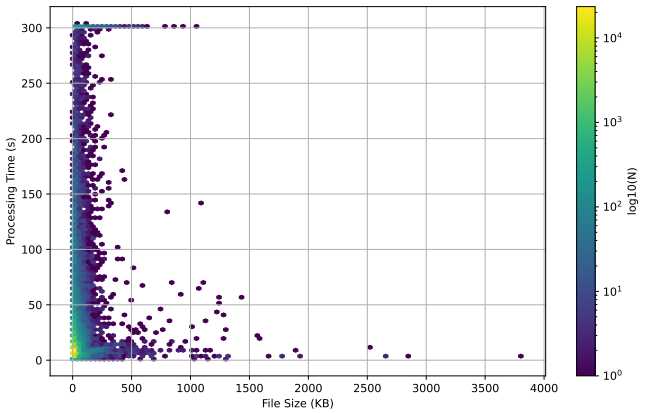


Figure 2: Runtime distribution for Phoebe across different input sizes

TABLE 4: Crashes & Timeouts: PHPDeobfuscator & Phoebe

OBF	Phoebe				PHPDeobfuscator			
	Syntax	🕐	Crash	$\sum$	Syntax	🕐	Crash	$\sum$
OBF 1	0	260	0	260	0	0	0	0
OBF 2	0	283	0	283	32 356	0	0	32 356
OBF 3	0	322	0	322	661	18	1617	2296
OBF 6	0	19	0	19	26	8	149	183
OBF 7	0	0	0	0	1401	10	48	1459
OBF 8	0	4	0	4	14	7	102	123
$\sum$	0	888	0	888	34 458	43	1916	36 417

TABLE 5: Similarity Metric comparing PHPDeobfuscator

Obf.	Obf.	Phoebe	PHPDeobfuscator
OBF 1	0.02 (σ=0.02)	0.76 (σ=0.35)	0.01 (σ=0.01)
OBF 2	0.00 (σ=0.00)	0.86 (σ=0.25)	0.00 (σ=0.00)
OBF 3	0.76 (σ=0.18)	0.88 (σ=0.16)	0.80 (σ=0.20)
OBF 6	0.96 (σ=0.13)	0.94 (σ=0.13)	0.91 (σ=0.19)
OBF 7	0.00 (σ=0.00)	0.83 (σ=0.32)	0.15 (σ=0.34)
OBF 8	0.23 (σ=0.23)	0.74 (σ=0.28)	0.69 (σ=0.30)

The clustering of the datapoints in the lower left quadrants shows, that Phoebe deobfuscates the majority of the data in less than 100 seconds. Additionally, the visualization also shows, that file size does not have strong correlation with runtime overall, as the majority of the datapoints are located in the leftmost column. The high densitity area to the the top of the figure represents Phoebe's timeouts which were present but overall limited.

Errors During Deobfuscation. Table 4 shows the number of syntactically incorrect results, timeouts, and crashes during deobfuscation. While Phoebe only returns syntactically valid files, PHPDeobfuscator frequently returns files with syntax errors. Especially for files obfuscated with OBF 2, PHPDeobfuscator always returns syntactically incorrect results. We analyzed the root cause and found PHPDeobfuscator failing to correctly unwrap a code pattern involving multiple nested evals specific to OBF 2. The number of erroneous results is less severe for the remaining obfuscators, but PHPDeobfuscator breaks some files during deobfuscation for all but OBF 1.

Crashes paint a similar picture, Phoebe does not crash on any sample, while PHPDeobfuscator crashes unexpectedly 1916 times, e.g., due to incorrect parsing of valid PHP code. Only when looking at timeouts Phoebe has a disadvantage, with 888 files timing out compared to 43 for PHPDeobfuscator. We want to note that we set a conservative timeout of five minutes to keep the total runtime manageable. If a human only has to analyze a single file, we believe that deobfuscation taking longer while returning a correct file is a favorable tradeoff. Taking all errors together, Phoebe outperforms PHPDeobfuscator by 97%, with only 888 issues compared to 36 417.

Similarity Measurement. Phoebe consistently outperforms PHPDeobfuscator across all obfuscators when it comes to restoring the original file Table 5 shows the mean similarity grouped by obfuscator. For each obfuscator Phoebe achieves a similarity above 0.70, while PHPDeobfuscator only does so for two and a third being close with an average of 0.69. However, for both of high average similarities of PHPDeobfuscator, the deobfuscated files already started with an average above or at 0.70, thus the overall improvement is either low (0.05) and in the second case even negative with a reduction of 0.05. This decrease is due to PHPDeobfuscator removing dead code and it changing the syntax of arrays, e.g., `array(1, "am")` becomes `array(0 => 1, 1 => "am")` after deobfuscation. Although semantically identical, the syntax differs when analyzing the ASTs to compute similarity. This issue also affects Phoebe, but only with a minor reduction of 0.02 as Phoebe only removes dead code.

Especially for OBF 1, 2, 7, which encode the entire program, PHPDeobfuscator has low (< 0.25) similarity to the original files; While Phoebe can successfully retrieve large parts of the original program, up to a mean of 0.86.

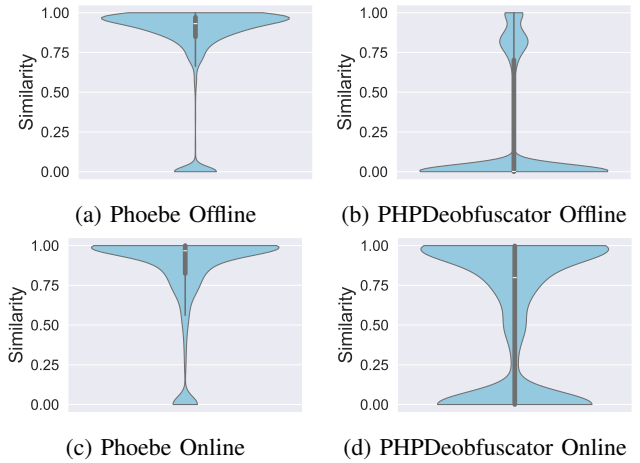
OBF 8 represents the only obfuscator with a low start similarity for which PHPDeobfuscator achieves noticeable similarity improvements. This is due to OBF 8 not encoding the entire program at once, but individual components such as strings and variables. This reduces the similarity considerably, as large parts of the program do not match the original, while the overarching program structure stays similar. PHPDeobfuscator can decode the majority of these obfuscated components and achieves a similarity of 0.69. However, Phoebe still outperforms it, achieving 0.74.

In addition to examining the similarity individually split by the deployed obfuscators, we also analyzed the similarity across all deobfuscated files. We leverage violin plots to showcase the similarity distribution of files, as shown in Figure 3, which displays the similarity for Phoebe and PHPDeobfuscator. We have split the visualization into two subplots per deobfuscator, one for the three offline obfuscators (OBF 1, 2, 3) that contribute over 89 800 files together. In contrast, the remaining three online obfuscators, namely OBF 6, 7, 8, only contribute roughly 5500 files. The split aims to ensure that the second, much smaller dataset, which may show a different distribution, stays visible. This is reflected in the Figures as well, while for Phoebe, Figure 3a and Figure 3c look similar, PHPDeobfuscator shows an entirely different distribution for Figure 3b compared to Figure 3d.

While both Phoebe and PHPDeobfuscator show a strong concentration of files with near-perfect similarity at the top of the violin plot for offline obfuscators, PHPDeobfuscator additionally exhibits a broader spread toward lower similarity values. This consistent presence across the entire plot suggests a higher proportion of partially or barely deobfuscated files. Moreover, PHPDeobfuscator shows a clear accumulation at the lower end of the similarity scale, indicating many files remain thoroughly obfuscated. This is consistent with the individual obfuscator results. PHPDeobfuscator fails to deobfuscate most files, with only a small subset reaching a similarity around 0.80, as indicated by a minor peak. In contrast, the majority of files processed by Phoebe achieve similarity scores well above this threshold.

TABLE 6: Complexity metrics for comparison with PHPDeobfuscator, displaying the difference from the original code. The closer a value is to zero, the closer it resembles the original file, visualized by the applied color coding.

Obf.	Cognitive						Cyclomatic					
	Obf.		Phoebe		PHPDeobfuscator		Obf.		Phoebe		PHPDeobfuscator	
OBF 1	−112.51	(σ=2.29)	−25.31	(σ=2.57)	−112.51	(σ=2.29)	−80.74	(σ=0.84)	−11.97	(σ=1.12)	−80.74	(σ=0.84)
OBF 2	−111.46	(σ=2.19)	−8.71	(σ=2.53)	−111.46	(σ=2.19)	−81.65	(σ=0.80)	−1.61	(σ=1.06)	−81.65	(σ=0.80)
OBF 3	243.86	(σ=2.51)	6.69	(σ=1.48)	59.88	(σ=1.95)	−0.07	(σ=0.71)	−0.18	(σ=0.71)	−2.06	(σ=0.70)
OBF 6	16.85	(σ=7.00)	3.98	(σ=6.87)	−1.83	(σ=6.69)	0.43	(σ=3.16)	0.39	(σ=3.16)	−1.70	(σ=3.12)
OBF 7	−83.53	(σ=4.52)	−11.46	(σ=6.01)	−83.53	(σ=4.52)	−57.70	(σ=2.62)	−9.17	(σ=3.07)	−57.70	(σ=2.62)
OBF 8	−10.24	(σ=4.77)	−10.30	(σ=4.77)	−11.18	(σ=4.68)	−5.29	(σ=2.34)	−5.32	(σ=2.34)	−5.90	(σ=2.32)



(a) Phoebe Offline (b) PHPDeobfuscator Offline

(c) Phoebe Online (d) PHPDeobfuscator Online

Figure 3: Similarity between the original and deobfuscated files for Phoebe and PHPDeobfuscator. Figure 3a and Figure 3b show the similarity across OBF 1, 2 and 3, while Figure 3c and Figure 3d do so for OBF 6, 7, and 8.

Cognitive & Cyclomatic Complexity. Examining the similarity provides insight into how well a deobfuscator can restore a file to its original state. However, even a non-similar result can still be useful if it makes understanding the code easier. We calculate the cognitive and cyclomatic complexity to capture this perspective and provide the differences from the original file values in Table 6.

Overall, Phoebe stays within a small margin to the original file (< 10) for both cognitive and cyclomatic complexity, except in three and one case, respectively. PHPDeobfuscator goes above this threshold in three cases for both metrics and reaches high (> 50) and very high (> 100) differences in four and three cases. Such a high difference shows that the presumably deobfuscated file can not resemble the original file and is likely unintelligible.

The results also mirror the success rate of PHPDeobfuscator in achieving similarity with OBF 1, 2, 7 being among the worst, and OBF 3, 6, 8 are among the best results, which is also reflected in the similarity results.

4.4. Comparison with UnPHP

We evaluated against UnPHP on the same hardware as PHPDeobfuscator. However, we have no insight into their hardware setup since UnPHP is an online tool. We ran our evaluation with a smaller data set to minimize strain on the service.

TABLE 7: Crashes & Timeouts of UnPHP and Phoebe

OBF	Phoebe				UnPHP			
	Syntax	🕑	Crash	∑	Syntax	🕑	Crash	∑
OBF 1	0	3	0	3	0	0	0	0
OBF 2	0	1	0	1	0	0	0	0
OBF 3	0	5	0	5	314	0	0	314
OBF 6	0	5	0	5	300	0	0	300
OBF 7	0	0	0	0	205	0	1	206
OBF 8	0	1	0	1	201	0	11	212
∑	0	15	0	15	1020	0	12	1032

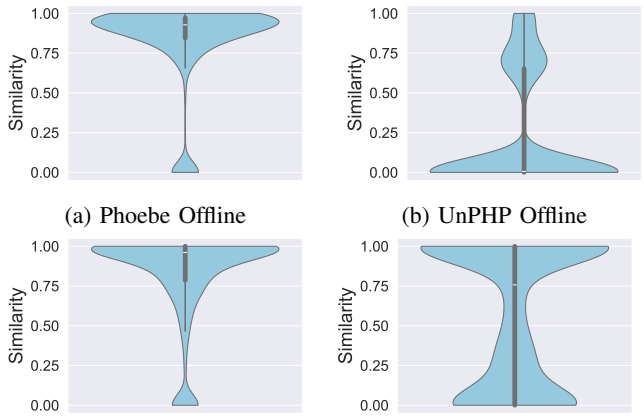
TABLE 8: Similarity Metric comparing UnPHP

Obf.	Obf.	Phoebe	UnPHP
OBF 1	0.02 (σ=0.02)	0.75 (σ=0.35)	0.01 (σ=0.02)
OBF 2	0.00 (σ=0.00)	0.89 (σ=0.21)	0.00 (σ=0.00)
OBF 3	0.76 (σ=0.17)	0.88 (σ=0.14)	0.74 (σ=0.21)
OBF 6	0.96 (σ=0.12)	0.94 (σ=0.13)	0.92 (σ=0.23)
OBF 7	0.00 (σ=0.00)	0.81 (σ=0.34)	0.53 (σ=0.49)
OBF 8	0.24 (σ=0.23)	0.75 (σ=0.28)	0.24 (σ=0.24)

Errors of UnPHP and Phoebe. Phoebe also outperforms UnPHP, incurring no syntactically incorrect results or errors, with Table 7 showing the errors that occur during deobfuscation of the dataset from Table 3. While UnPHP can handle every obfuscator to some degree, the results show it struggling to deobfuscate all of OBF 3, 6, 7, 8. For each of them, UnPHP returns at least 40% syntactically incorrect results. Meanwhile, Phoebe again exhibits a slightly higher number of timeouts, totaling 15.

Similarity Measurement. Phoebe outperformed UnPHP across all obfuscators in achieving a high mean similarity with the original file. While UnPHP only achieves a similarity higher than 0.75 for a single deobfuscator, namely OBF 6, Phoebe did so for all, as shown in Table 8. Similar to the offline evaluation, UnPHP achieves a value higher than 0.75 only for an obfuscator that already started with a very high similarity (0.96) with the next closest one being OBF 3 where it achieves a mean similarity of 0.74 a decrease of 0.02.

Again, similar to the comparison with PHPDeobfuscator, UnPHP is unable to deobfuscate code generated by OBF 1, 2 and does not achieve similarity results above 0.01. However, for OBF 7, UnPHP achieves 0.53 similarity while PHPDeobfuscator did not rise above 0.15 mean similarity. This obfuscator is the only one for which UnPHP can increase the mean similarity by more than 0.25.



(a) Phoebe Offline (b) UnPHP Offline

(c) Phoebe Online (d) UnPHP Online

Figure 4: Similarity between the original and deobfuscated files for Phoebe and UnPHP. Figure 4b show the similarity across OBF 1, 2 and 3, while Figure 4c and Figure 4d depit it for OBF 6, 7 and 8. Figure 4a and .

We again visualize the similarity distribution as violin plots in Figure 4. As we saw a different pattern by splitting the plot among the online and offline obfuscator axes, we kept the split in place for this set of plots, despite the datasets having the same size. Looking at the distribution for UnPHP, we can see that the results resemble the comparison with PHPDeobfuscator. For Phoebe, the majority of the figure resides above the 0.90 line. In contrast, for UnPHP, only a small portion achieves very high values, followed by a broader distribution that tapers into a substantial accumulation at the lower end of the plot, again indicating a significant number of unchanged files.

Cognitive & Cyclomatic Complexity. Table 9 shows the cognitive and cyclomatic complexity differences between the original and obfuscated and deobfuscated files. The results are similar to our prior comparison with PHPDeobfuscator as UnPHP stays above 10 difference for five obfuscators with cognitive similarity, and only goes above that value for cyclomatic complexity in three cases. However, these three cases already start at a very low base value, highlighting how UnPHP barely simplifies code. The values for cognitive complexity show that while Phoebe can improve complexity by about or above 100 for two obfuscators, UnPHP only achieves an improvement over 50 in a single case, with improvements below 5 for all others.

The picture for cyclomatic complexity is the same: Phoebe consistently reduces the difference to the original file by more than 50 where possible, which UnPHP does not. UnPHP only reaches values below 10 for obfuscated files also starting in that order of magnitude. The only larger improvement (> 25) is achieved for OBF 7, identical to the improvement patterns for cognitive complexity.

4.5. Efficacy on Real WebShells

To highlight the efficacy of Phoebe for security evaluations we conducted an experiment based on 40 webshells from a public GitHub repository [41]. As explained in Section 2, webshells commonly employ obfuscation and are, thus, a prime area of application for deobfuscation tools.

As we do not have the original versions of the webshells prior to any obfuscation, we cannot conduct the same evaluation approach as before but have to adapt. For the complexity metrics, we analyze the obfuscated and deobfuscated versions to determine the impact of deobfuscation on code complexity. Given that we cannot assess the achieved code similarity without having access to the original code we replace the similarity analysis with a manual analysis in which two researchers inspect the code changes and assess the impact. The focus of this assesment is on whether one can deduce the (potentially malicious) behavior of the files. In case of disagreement between the two experts a third expert weighs in and casts the deciding vote.

Cognitive & Cyclomatic Complexity. Table 12 in the Appendix shows the complexity metric values for all webshells. The evaluation reveals that if instruction patterns leading to complexity are recognizable in the obfuscated version, they almost always remain the same in the deobfuscated version. This is because those webshells do not employ GOTO statements to complicate the control flow. Furthermore, only individual values are encoded, so that resolving these encodings does not decrease the overall complexity. However, two webshells (wwwolf-php-webshell.php, K2LL33D-SHELL.php) are exceptions to this patterns as their obfuscation heavily relies on GOTO statements. This resulted in Phoebe reducing the cognitive complexity from 292 to 70 and from 3301 to 813. Another exception to our initial observation are the webshells that use encoding. Phoebe reconstructs large parts of the code, significantly raising the complexity from an initial value of 0.

Finally, we have two webshells (root-index.php, 4RC.php) that completely break with our complexity analysis. The first webshell does not contain any PHP, only HTML and JavaScript, so that no complexity can be measured and is out scope of a PHP deobfuscation tool. In the second case, we can partially restore the code, but it remains as a string parameter in an eval statement and therefore a further analysis of the complexity is not meaningful.

Manual Review. Replacing the code similarity analysis, two researchers manually reviewed all 40 webshells to determine whether they exhibited any discernible malicious behavior prior and post deobfuscation. Table 12 shows the corresponding results. One webshell (root-index.php) was excluded because it did not contain PHP code. Of the remaining 39 webshells, one (b374k-Shell.php) was already recognizable as malicious in its obfuscated form. The malicious behavior of the other 38 webshells was not humanly discernable prior to deobfuscation. After deobfuscation, malicious behavior was discernable in all webshells.

TABLE 9: Complexity metrics for comparison with UnPHP, displaying the difference from the original code. The closer a value is to zero, the closer it resembles the original file, visualized by the applied color coding.

Obf.	Cognitive						Cyclomatic					
	Obf.		Phoebe		UnPHP		Obf.		Phoebe		UnPHP	
OBF 1	−109.93	(σ=10.13)	−15.18	(σ=14.03)	−109.93	(σ=10.13)	−79.78	(σ=7.29)	−7.60	(σ=10.27)	−79.78	(σ=7.29)
OBF 2	−125.95	(σ=12.71)	−9.25	(σ=16.26)	−125.95	(σ=12.71)	−88.01	(σ=6.10)	−1.90	(σ=7.94)	−88.01	(σ=6.10)
OBF 3	254.06	(σ=19.05)	6.62	(σ=11.67)	251.34	(σ=19.11)	−0.03	(σ=5.11)	−0.48	(σ=5.09)	−0.49	(σ=5.11)
OBF 6	14.20	(σ=19.39)	1.91	(σ=19.25)	9.98	(σ=19.38)	0.52	(σ=7.84)	0.43	(σ=7.84)	−2.10	(σ=7.84)
OBF 7	−76.37	(σ=6.84)	−8.35	(σ=9.57)	−25.86	(σ=9.86)	−53.20	(σ=3.51)	−6.61	(σ=4.86)	−24.01	(σ=4.77)
OBF 8	−12.09	(σ=7.69)	−12.12	(σ=7.69)	−12.09	(σ=7.69)	−5.94	(σ=3.75)	−5.95	(σ=3.75)	−5.94	(σ=3.75)

5. Discussion

Studying deobfuscation enabled us to assess obfuscators' capabilities, leverage multiple metrics to quantify the impact of transformations on code, compare different deobfuscators, and develop Phoebe based on observed obfuscation patterns.

5.1. Obfuscator Strengths and Weaknesses

Our analysis covered ten obfuscators using nine patterns, revealing their strengths, limitations, and applicability.

High Variety of Available Transformations. The number of employed patterns varies widely between the different obfuscators. Identifier Mangling and Encoding are the most popular patterns, with eight obfuscators employing it. The least popular patterns are If-Negation and Loop-Hiding, with only YAK Pro using either. Naneau uses the fewest patterns, i.e., two, whereas several obfuscators employ six patterns.

Each transformation affects obfuscated files differently. Identifier Mangling alters only variable and function names, whereas Encoding significantly restructures the code by hiding sections. When combined with Dynamic Code Execution, this often reduces code to a single opaque blob, concealing both data and logic, hindering analysis.

Our observations identify YAK Pro as the most challenging obfuscator. It uses encoding to obscure values and heavily alters code structure by inserting conditionals, mixing basic blocks, and deconstructing loops. While reversing conditionals is straightforward, block mixing and loop deconstruction are difficult to undo manually, rendering the code unintelligible without specialized tools. These changes are reflected in cognitive complexity. YAK Pro increases complexity by 235 on average. Thus, users must carefully select obfuscation tools based on their needs. Many tools are available for lightweight obfuscation that may not resist manual inspection. However, if the obfuscation is supposed to be thorough and not easily reversible, only a handful of obfuscators go beyond hiding the code in an encoded string.

Take-Away 1: Obfuscators starkly differ in capabilities and supported obfuscation patterns, YAK Pro being the most capable.

Limitations, Runtime Exceptions, and Subtle Errors. Out of the ten obfuscators, four could not be fully evaluated: two used CAPTCHAs, preventing automation, one returned a 526 HTTP error, and one failed to process classes, a core PHP feature, limiting its applicability.

Five obfuscators threw exceptions on 2% to 20% of inputs, and syntax errors appeared for 3% to nearly 50% of obfuscated files, raising concerns about their reliability. For example, PHP-Einfach causes name collisions due to PHP's case-insensitivity (e.g., `fun` vs. `fUN`), and PHP Obfuscator misplaces critical directives like `namespace` or `declare`, which must appear at the beginning of a file.

Such issues complicate deobfuscation, as syntax errors introduced by obfuscators often surface only after partially reversing outer encoding or dynamic execution layers.

Take-Away 2: Obfuscators frequently introduce subtle errors.

5.2. Meaningfulness of the Different Metrics

We evaluated three metrics to assess the impact of obfuscation on source code and the extent to which deobfuscation can reverse the changes. However, we noticed differences in their meaningfulness across obfuscation patterns.

Similarity as a Metric. Code similarity to its original version is an intuitive way to assess (de)obfuscation impact. Encoding and Dynamic Code Execution heavily restructure code by moving logic into strings, reducing similarity. These patterns are reversible if properly identified and supported, allowing deobfuscators supporting these patterns to achieve high similarities. However, perfect similarity is not always achievable. Our manual analysis identifies two main reasons why Phoebe's average similarity falls below 1.0. First, deobfuscation may inadvertently optimize code—for example, through constant propagation that inlines values not originally inlined. Second, obfuscators insert functions used only by obfuscated code; after deobfuscation, these remain unused but are retained, as a deobfuscator cannot reliably distinguish them from externally referenced utilities. This uncertainty prevents safe removal and slightly reduces similarity. Although intuitive, similarity has limitations. For example, YAK Pro, the most complex obfuscator, still yields a relatively high similarity of 0.76. It modifies the control flow, e.g., removing loops and inverting conditions while leaving small code fragments intact. These unchanged fragments align with the original AST, inflating similarity despite the code being unreadable without specialized tools. Thus, similarity can underestimate the actual complexity of obfuscation.

Take-Away 3: Code similarity as a metric does not always reflect the true degree of obfuscation.

Cognitive vs. Cyclomatic Complexity. Besides similarity, we use cognitive and cyclomatic complexity to determine how obfuscation affects code comprehensibility.

Cognitive complexity is particularly useful for analyzing logic-based obfuscation and complements similarity. For example, YAK Pro retains a similarity of 0.76 but increases cognitive complexity by 235, indicating a substantial rise in mental effort to understand the obfuscated code, confirmed through manual analysis.

Complexity changes vary across obfuscators. Negative values suggest reduced complexity, while positive ones indicate increases. OBF 1, 2, 7 show reductions in both metrics due to Encoding and Dynamic Code Execution. OBF 7 reduces files to a single line, `eval(gzinflate(base64_decode(..)))`, eliminating most control flow. OBF 2 follows similar patterns, replacing structure with runtime decoding logic, which results in low complexity scores. In contrast, the files obfuscated with OBF 1 consist of boilerplate code introduced by the obfuscator, which is used to decode the original code during runtime, but follow a similar pattern with dynamic code execution.

Cyclomatic complexity, however, proves less reliable. Despite YAK Pro's substantial structural changes, its cyclomatic complexity remains nearly constant. This metric only counts branches and does not capture `goto` statements, meaning techniques like Basic Block Mixing, though highly obfuscating, do not affect its value. Cognitive complexity does not share these shortcomings and captures such transformations more accurately, as seen in Table 6 and Table 9 when considering the values for YAK Pro, i.e., OBF 3.

Thus, while both metrics provide valuable insights, cognitive complexity more effectively reflects the true extent of obfuscation.

Take-Away 4: Cognitive complexity is more meaningful than cyclomatic complexity to assess (de)obfuscation.

5.3. Deobfuscation Impact

While evaluating existing deobfuscators and developing our own, we noticed peculiarities concerning the reliability of existing deobfuscation solutions, the impact that handling single obfuscation patterns can have, and how deobfuscation as a whole can improve code comprehensibility.

Deobfuscation Correctness. We noticed that the previous state-of-the-art deobfuscators fail to handle common obfuscation patterns. They either return files with syntax errors at varying frequencies or have no impact on the obfuscation. This increases the effort for an analyst when using either of them, as syntactically incorrect files degrade features for integrated development environments, e.g., preventing advanced code navigation, and entirely prevent automatic analysis by tools relying on correct parsing, e.g., [18]. The work by Starov et al. [38], who used UnPHP to deobfuscate PHP web shells, directly highlights this issue. They report UnPHP returning high amounts of broken files, requiring considerable effort to fix, if possible, for the authors. Consequently, a deobfuscator must ensure that it only returns syntactically valid files; otherwise, its usability is severely limited.

Take-Away 5: Previous deobfuscators frequently destroy files.

Deobfuscator Ability to Handle Patterns. Except Phoebe, none of the deobfuscators we evaluated could handle all obfuscation patterns we observed in our analysis. While PHPDeobfuscator achieves a difference in similarity for at least four obfuscators, UnPHP only does so for three.

This significantly impacts the results, which is reflected in all metrics and the number of errors. As the patterns often occur in combination, for example, Data Scattering and Encoding, an unresolved pattern can also influence the results for another pattern. Even if the encoding can be resolved, it is not possible if it is not distinguishable due to Data Scattering. Thus, deobfuscators need to stay up to date to remain useful, as even one unsupported pattern severely impacts the results.

Take-Away 6: Failure to handle a single obfuscation pattern can render a debofuscator effectively useless.

Impact of Deobfuscation on Code Complexity. We manually investigated various code examples to understand the differences between cognitive and cyclomatic complexity. Here, we noticed how deobfuscation also contributes to code improvement. For example, removing dead code reduces complexity compared to the original code. Frequently, the dead code we encountered even contained complex program logic with loops and `if` statements, which explains the noticeable reduction in complexity.

Take-Away 7: Deobfuscation can inadvertently improve code.

5.4. Limitations & Future Work

Phoebe is subject to the same limitations as any other deobfuscation approach. Namely, some information, such as method, class, or variable names, cannot be recovered once lost during obfuscation. Furthermore, Phoebe cannot determine whether specific structures, such as missing line breaks or control flow handling with `goto` statements, were intended by the developer or introduced during obfuscation. We consider these limitations negligible, as they ultimately improve the readability of the resulting code. Similarly, our constant propagation sometimes extends beyond the original file.The only drawback of Phoebe compared to UnPHP and PHPDeobfuscator is the higher number of timeouts. We believe the complete absence of runtime crashes and invalid files returned by Phoebe, as well as greatly improved similarity results, offsets this limitation. Similarly, optimizing the implementation of Phoebe is certainly possible, but was not the focus of this work. As a human analyst often spends a considerable amount of time with a file to analyze, deobfuscation with Phoebe taking longer is offset by Phoebe being able to retrieve more of the original file, reducing manual deobfuscation effort. Finally, Phoebe, as

well as any other obfuscator, is not resistant against yet to be developed obfuscation techniques but can only address known transformations. Currently, Phoebe supports a wide range of APIs commonly used for obfuscation purposes. With respect to compression, Phoebe implements support for `gzuncompress` and `gzinflate`. In terms of encoding, Phoebe supports several built-in methods, including `rot13`, `base64`, `htmlspecialchars_decode`, and `hex2bin`. In addition, it provides support for a variety of string manipulation functions, such as `substr`, `strrev`, `ord`, `chr`, `str_replace`, `str_ireplace`. For dynamic code evaluation, Phoebe handles the **`eval`** construct, provided that the code can be safely inlined without altering program semantics, for example, in cases involving dynamic method invocations acting as input for another eval or method call. Nevertheless, compression or encoding algorithms not explicitly supported cannot be resolved. To mitigate this limitation, Phoebe adopts a modular design that facilitates the integration of new deobfuscation transformations, thereby ensuring extensibility as new algorithms or obfuscation techniques emerge.

6. Related Work

We grouped related work into three areas: obfuscation detection, deobfuscation, and deobfuscation usage.

Obfuscation Detection. Detecting obfuscation remains an active area of research. Bacci et al. [3] combine static analysis with machine learning to detect obfuscation in Android apps. PowerShell has been a major focus, with various classifiers used to detect obfuscated scripts [14, 15, 25]. Furthermore, combined approaches using AST-based static analysis and machine learning have been explored [33]. Fass et al. [11] detect obfuscated JavaScript by applying a random forest classifier to AST-derived features, whereas Sarker et al. [35] use an instrumented browser, focusing on analyzing API usage. However, detection alone is insufficient, as obfuscation can also serve legitimate purposes like software protection, watermarking, or tamper-proofing [7]. Deobfuscation is necessary to determine malicious intent, such as hidden backdoors, a gap we address with Phoebe.

Deobfuscation. Many works address virtualization-based obfuscation [8, 23, 34], but these techniques do not necessarily transfer to higher-level scripting languages. Within the domain of scripting languages, several works focus on PowerShell. Li et al. [24] use a dynamic approach to track execution and recover obfuscated instructions. Chai et al. [5] enhance AST analysis with execution steps to reconstruct obfuscated fragments. Ugarte et al. [39] combine static and dynamic methods via code instrumentation, while Li et al. [25] emulate obfuscated AST subtrees in PowerShell sessions. For JavaScript, Herrera [16], apply AST transformations (e.g., constant folding, propagation, dead code removal), but lack support for built-in or custom decoding functions, limiting its applicability to our setting. Also, JavaScript lacks `goto`, greatly limiting obfuscation to the logical structure. Weißer et al. [44] study PHP bytecode obfuscation, analyze tools like Zend Guard and IonCube, and present a decompiler. However, bytecode obfuscation involves different challenges and obfuscation methods, since it is a binary format and not a higher-level scripting language. The most PHP-related work is by Naderi-Afooshteh et al. [28], who dynamically deobfuscate PHP scripts by executing them in a controlled environment, producing multiple programs as a result. Their limited evaluation and lack of artifact regrettably make a comparison impossible. In contrast, Phoebe is entirely static and deterministic, setting it apart from dynamic methods.

Deobfuscation Usage. Deobfuscation is often used as a preprocessing step for further analysis. In PHP program analysis, especially UnPHP is frequently deployed to deobfuscate programs without evaluating its effectiveness. Huang et al. [17] and Zhang et al. [46] use UnPHP for deobfuscation to extract features from the deobfuscated code. Lastly, Starov et al. [38] utilizes UnPHP to deobfuscate PHP webshells for static analysis, but question its reliability due to many broken files requiring manual fixes. Similarly, Naderi-Afooshteh et al. [29] deliberately avoid UnPHP after observing inconsistent, non-deterministic results. These issues highlight the need for reliable deobfuscation. Our work addresses this by introducing Phoebe, a deterministic, syntax-error-free deobfuscator that provides a robust foundation for further analysis of malicious PHP code.

7. Conclusion

Obfuscation renders the code entirely incomprehensible to a human analyst, while preserving the semantics of the original file, presenting a common technique to disguise the true nature of malicious code. To counter this and assist humans in analyzing obfuscated code, deobfuscators are required. In this work, we first study the state of PHP obfuscation by analyzing ten obfuscators for the code transformations they employ. We identify nine such transformations, ranging from rather simplistic, such as changing identifiers to randomly chosen strings, to fairly complex, such as obfuscating the logical structure of the code. Based on these insights, we built Phoebe, a PHP deobfuscator capable of reversing all encountered obfuscation techniques, except those that remove information, such as names, during obfuscation. We evaluate Phoebe on 95541 files from large open-source PHP projects, which we first obfuscate using six of those obfuscators. We then compare their deobfuscation results with those of the state-of-the-art PHP deobfuscators. To assess the efficacy of deobfuscation, we evaluate four metrics: correctness of the output, code similarity, and two code complexity metrics. Phoebe outperforms both other deobfuscators by a considerable margin.

Acknowledgments

Funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972.

References

[1] Php code obfuscation. https://php-obf.com/. Accessed: 2025-04-03.

[2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson Education, London, 2006.

[3] Alessandro Bacci, Alberto Bartoli, Fabio Martinelli, Eric Medvet, and Francesco Mercaldo. Detection of obfuscation techniques in android applications. In *Proc. of the International Conference on Availability, Reliability and Security (ARES)*, 2018.

[4] G Ann Campbell. Cognitive complexity-a new way of measuring understandability. *SonarSource SA*, 10, 2018.

[5] Huajun Chai, Lingyun Ying, Haixin Duan, and Daren Zha. Invoke-deobfuscation: Ast-based and semantics-preserving deobfuscation for powershell scripts. In *Proc. of the Conference on Dependable Systems and Networks (DSN)*, 2022.

[6] Pipsomania Co. Best php obfuscator. http://www.pipsomania.com/best_php_obfuscator.do. Accessed: 2025-04-03.

[7] Christian Collberg and Clark Thomborson. Watermarking, tamper-proofing, and obfuscation - tools for software protection. *IEEE Transactions on Software Engineering*, 28, 2002.

[8] Kevin Coogan, Gen Lu, and Saumya Debray. Deobfuscation of virtualization-obfuscated software: a semantics-based approach. In *Proc. of the ACM Conference on Computer and Communications Security (CCS)*, 2011.

[9] Marco Cova, Christopher Kruegel, and Giovanni Vigna. There is no free phish: An analysis of" free" and live phishing kits. *WOOT Conference on Offensive Technologies*, 2008.

[10] Rodrigue Dessaints. Php obfuscator. https://php-minify.com/php-obfuscator/index.php. Accessed: 2025-04-03.

[11] Aurore Fass, Robert P. Krawczyk, Michael Backes, and Ben Stock. Jast: Fully syntactic detection of malicious (obfuscated) javascript. In *Proc. of the Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA)*, 2018.

[12] Maurice Fonk. Naneau php obfuscator. https://github.com/naneau/php-obfuscator. Accessed: 2025-04-03.

[13] The PHP Documentation Group. PHP: eval - Manual. https://www.php.net/manual/en/function.eval.php#refsect1-function.eval-returnvalues. Accessed: 2025-04-22.

[14] Danny Hendler, Shay Kels, and Amir Rubin. Detecting malicious powershell commands using deep neural networks. In *Proc. of the ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, 2018.

[15] Danny Hendler, Shay Kels, and Amir Rubin. Amsi-based detection of malicious powershell code using contextual embeddings. In *Proc. of the ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, 2020.

[16] Adrian Herrera. Optimizing away javascript obfuscation. In *Proc. of the IEEE International Conference on Source Code Analysis & Manipulation (SCAM)*, 2020.

[17] Weiqing Huang, Chenggang Jia, Min Yu, Kam-Pui Chow, Jiuming Chen, Chao Liu, and Jianguo Jiang. Enhancing the feature profiles of web shells by analyzing the performance of multiple detectors. In *Advances in Digital Forensics XVI*, volume 589. 2020.

[18] Feras Al Kassar, Giulia Clerici, Luca Compagna, Davide Balzarotti, and Fabian Yamaguchi. Testability Tarpits: the Impact of Code Patterns on the Security Testing of Web Applications. In *Network and Distributed System Security Symposium (NDSS)*, 2022.

[19] Ranjita Pai Kasturi, Jonathan Fuller, Yiting Sun, Omar Chabklo, Andres Rodriguez, Jeman Park, and Brendan Saltaformaggio. Mistrust plugins you must: A {Large-Scale} study of malicious plugins in {WordPress} marketplaces. In *USENIX Security Symposium*, 2022.

[20] Isma Khairunisa and Herman Kabetta. Php source code protection using layout obfuscation and aes-256 encryption algorithm. In *International Workshop on Big Data and Information Security (IWBIS)*, 2021.

[21] Pascal Kissian. Yak pro. https://github.com/pk-fr/yakpro-po. Accessed: 2025-04-03.

[22] Simon Koch, David Klein, and Martin Johns. The fault in our stars: An analysis of GitHub stars as an importance metric for web source code. In *Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb)*, San Diego, USA, 2024.

[23] Patrick Kochberger, Sebastian Schrittwieser, Stefan Schweighofer, Peter Kieseberg, and Edgar Weippl. Sok: Automatic deobfuscation of virtualization-protected applications. In *Proc. of the International Conference on Availability, Reliability and Security (ARES)*, 2021.

[24] Ruijie Li, Chenyang Zhang, Huajun Chai, Lingyun Ying, Haixin Duan, and Jun Tao. Powerpeeler: A precise and general dynamic deobfuscation method for powershell scripts. In *Proc. of the ACM Conference on Computer and Communications Security (CCS)*, 2024.

[25] Zhenyuan Li, Yan Chen, Qi Alfred Chen, Tiantian Zhu, Chunlin Xiong, and Hai Yang. Effective and light-weight deobfuscation and semantic-aware attack detection for powershell scripts. In *Proc. of the ACM Conference on Computer and Communications Security (CCS)*, 2019.

[26] T.J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 1976.

[27] Mobilefish. Simple online php obfuscator. https://www.mobilefish.com/services/php_obfuscator/php_obfuscator.php. Accessed: 2025-04-03.

[28] Abbas Naderi-Afooshteh, Yonghwi Kwon, Anh Nguyen-Tuong, Mandana Bagheri-Marzijarani, and Jack W Davidson. Cubismo: Decloaking server-side malware via cubist program analysis. In *Proc. of the Annual Computer Security Applications Conference*

(ACSAC), 2019.

[29] Abbas Naderi-Afooshteh, Yonghwi Kwon, Anh Nguyen-Tuong, Ali Razmjoo-Qalaei, Mohammad-Reza Zamiri-Gourabi, and Jack W. Davidson. Malmax: Multi-aspect execution for automated dynamic web server malware analysis. In *Proc. of the ACM Conference on Computer and Communications Security (CCS)*, 2019.

[30] Nikita Popov. Php parser. https://github.com/nikic/PHP-Parser. Accessed: 2025-04-22.

[31] Nils Reimers. Php obfuscator. https://www.php-einfach.de/diverses/php-code-verschluesselung-php-obfuscator/. Accessed: 2025-04-03.

[32] Werner Rumpeltesz. Gaijn php obfuscator. https://www.gaijin.at/en/tools/php-obfuscator. Accessed: 2025-04-03.

[33] Gili Rusak, Abdullah Al-Dujaili, and Una-May O'Reilly. Ast-based deep learning for detecting malicious powershell. In *Proc. of the ACM Conference on Computer and Communications Security (CCS)*, 2018.

[34] Jonathan Salwan, Sébastien Bardin, and Marie-Laure Potet. Symbolic deobfuscation: From virtualized code back to the original. In *Proc. of the Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA)*, 2018.

[35] Shaown Sarker, Jordan Jueckstock, and Alexandros Kapravelos. Hiding in plain site: Detecting javascript obfuscation through concealed browser api usage. In *Internet Measurement Conference (IMC)*, 2020.

[36] Simon816. Phpdeobfuscator. https://github.com/simon816/PHPDeobfuscator. Accessed: 2025-04-23.

[37] Pierre-Henry Soria. Ph7 php obfuscator. https://github.com/pH-7/Obfuscator-Class. Accessed: 2025-04-03.

[38] Oleksii Starov, Johannes Dahse, Syed Sharique Ahmad, Thorsten Holz, and Nick Nikiforakis. No honor among thieves: A large-scale analysis of malicious web shells. In *The Web Conference*, 2016.

[39] Denis Ugarte, Davide Maiorca, Fabrizio Cara, and Giorgio Giacinto. Powerdrive: Accurate de-obfuscation and analysis of powershell malware. In *Proc. of the Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA)*, 2019.

[40] Unknown. Unphp. https://www.unphp.net/, . Accessed: 2025-04-23.

[41] Unknown. Webshell. https://github.com/php-shell-list/WebShell, . Accessed: 2025-09-10.

[42] W3Techs. Usage statistics of php for websites. https://w3techs.com/technologies/details/pl-php, . Accessed: 2025-04-22.

[43] W3Techs. Usage statistics and market share of wordpress. https://w3techs.com/technologies/details/cm-wordpress, . Accessed: 2025-04-22.

[44] Dario Weißer, Johannes Dahse, and Thorsten Holz. Security analysis of php bytecode protection mechanisms. In *International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2015.

[45] Weidi Zhang. Smart-php-obfuscator. https://github.com/weidizhang/smart-php-obfuscator. Accessed: 2025-04-03.

[46] Zijian Zhang, Meng Li, Liehuang Zhu, and Xinyi Li. SmartDetect: A Smart Detection Scheme for Malicious Web Shell Codes via Ensemble Learning. In *Smart Computing and Communication*, volume 11344. 2018.

Appendix A.
Availability

Phoebe, together with test data and usage instructions, is publicly available at https://github.com/ias-tubs/Phoebe.

Appendix B.
Dataset Filtering

This section provides a detailed overview of the errors encountered during the obfuscation and evaluation, which resulted in the exclusion of certain files from the final analysis. Table 10 and Table 11 show the error distributions for the comparison with PHPDeobfuscator and UnPHP, respectively. The columns in each table correspond to distinct error types encountered throughout different phases of the dataset creation process. Syntax errors refer to files in the original dataset that were syntactically invalid and thus excluded from further processing. Obfuscation errors (OB errors) denote failures caused by the obfuscation tools themselves, such as crashes. OB syntax errors represent cases where the output of the obfuscation was syntactically incorrect. Additionally, timeouts arising during the similarity analysis phase are also reported. By subtracting all of these error cases from the original dataset, we obtain the final set of valid, successfully obfuscated files, which we refer to as the obfuscated dataset.

TABLE 10: Overview of the number and types of errors that occurred within the dataset used for the comparison with PHPDeobfuscator

OBF	Original	Syntax Errors	OB Errors	OB Syntax Errors	Similarity Errors	Obfuscated
OBF 1	33 000	47	1474	710	0	30 769
OBF 2	33 000	47	597	0	0	32 356
OBF 3	33 000	47	3388	1930	927	26 708
OBF 6	3000	3	607	101	23	2266
OBF 7	3000	3	25	907	0	2065
OBF 8	3000	3	0	1453	167	1377

TABLE 11: Overview of the number and types of errors that occurred within the dataset used for the comparison with UnPHP

OBF	Original	Syntax Errors	OB Errors	OB Syntax Errors	Similarity Errors	Obfuscated
OBF 1	500	0	0	0	0	500
OBF 2	500	0	0	0	0	500
OBF 3	500	0	0	2	24	474
OBF 6	500	0	0	0	9	491
OBF 7	500	0	0	0	0	500
OBF 8	500	0	0	0	44	456

Appendix C.
Webshells

TABLE 12: Complexity metrics for comparison of obfuscated and deobfuscated Webshells. The last column indicates whether malicious behavior was observed in the original webshell (❑) or only after deobfuscation (☑).

File	Obfuscated		Deobfuscated		Malicious
	Cognitive	Cyclomatic	Cognitive	Cyclomatic	
terminal.php	30.00	13.00	30.00	13.00	☑
Simple-PHP-Shell.php	225.00	78.00	225.00	78.00	☑
PHP-SSH-Shell.php	166.00	70.00	166.00	70.00	☑
keisatsu_shell.php	246.00	139.00	246.00	139.00	☑
1945-Mini-Shell.php	0.00	0.00	142.00	95.00	☑
Prvi8-Ir-Web-Shell.php	105.00	59.00	105.00	59.00	☑
nsscmdshell.php	71.00	40.00	71.00	40.00	☑
0xShell.php	384.00	199.00	384.00	199.00	☑
Yavuzlar-WebShell.php	0.00	0.00	257.00	125.00	☑
MWS-Shell.php	0.00	0.00	481.00	253.00	☑
PHP-Web-Shell-v1.php	0.00	0.00	96.00	62.00	☑
Kumasia.php	0.00	0.00	95.00	59.00	☑
p0wny.php	91.00	44.00	91.00	44.00	☑
WSO-Shell.php	916.00	523.00	916.00	523.00	☑
PHP-Web-Shell-v2.php	0.00	0.00	96.00	62.00	☑
RootKit.php	112.00	54.00	112.00	54.00	☑
R00t-Shell_PHP_Pwner.php	0.00	0.00	9.00	8.00	☑
root-index.php	0.00	0.00	0.00	0.00	✗
indoxploit.php	1341.00	400.00	1341.00	400.00	☑
bheh_php_shell.php	230.00	161.00	230.00	161.00	☑
BypassServ.php	144.00	68.00	144.00	68.00	☑
b374k-Shell.php	2687.00	816.00	2687.00	816.00	❑
P1ckl3d-Web-Shell.php	36.00	28.00	36.00	28.00	☑
pas_fork.php	0.00	0.00	5050.00	1674.00	☑
IndoSec-sHell.php	891.00	434.00	891.00	434.00	☑
DCSC-PHP-Shell.php	248.00	88.00	248.00	88.00	☑
WSO.php	903.00	513.00	904.00	514.00	☑
Gelxy-Mini-Shell.php	175.00	83.00	175.00	83.00	☑
Gecko-Shell.php	668.00	208.00	668.00	208.00	☑
Mini-Shell.php	341.00	151.00	341.00	151.00	☑
MARIJUANA.php	0.00	0.00	207.00	97.00	☑
Root-Shell-CMD.php	0.00	0.00	25.00	19.00	☑
Ani-Shell.php	2635.00	273.00	2635.00	273.00	☑
wwwolf-php-webshell.php	292.00	47.00	70.00	47.00	☑
K2LL33D-SHELL.php	3301.00	493.00	813.00	490.00	☑
R57-Shell.php	1987.00	1074.00	1987.00	1074.00	☑
C99-Shell.php	3802.00	894.00	3802.00	894.00	☑
adminer.php	7958.00	3560.00	7958.00	3560.00	☑
adminer-4.16.0.php	7930.00	3517.00	7930.00	3517.00	☑
4RC.php	0.00	0.00	0.00	0.00	☑